

Università degli Studi di Udine

Dipartimento di Scienze Matematiche, Informatiche e Fisiche

Progetto di Laboratorio di ASD

Bombardella Davide, 167021
bombardella.davide@spes.uniud.it

La Guardia Aurora, 166531
laguardia.aurora@spes.uniud.it

Pascolat Anna, 169047

pascolat.anna@spes.uniud.it

6 Giugno 2025

Indice

1 Introduzione **2**

2 Algoritmi di Ordinamento **2**
2.1 *Quick Sort* 2
2.2 *Quick Sort 3-way* 2
2.3 *Counting Sort* 3
2.4 *Merge Sort* 3

3 Algoritmi di Misurazione **4**
3.1 Resolution 4
3.2 Measure 4

4 Rappresentazione Grafica dei Risultati **5**

5 Conclusioni **7**

1 Introduzione

Il progetto assegnato richiede l'implementazione di alcuni algoritmi di ordinamento e la relativa analisi sperimentale. In particolare, si analizzano e confrontano i seguenti algoritmi: *Quick Sort*, *Quick Sort 3-way*, *Counting Sort* e *Merge Sort*; rispettivamente i primi tre scelti dal docente e l'ultimo scelto dal gruppo. La scelta di *Merge Sort* è stata motivata dalla volontà di confrontare algoritmi aventi costi computazionali differenti. Se fosse stato scelto, ad esempio, *Insertion Sort* (più semplice da implementare), avremmo ottenuto due algoritmi con la stessa complessità di $\Theta(n^2)$, limitando così la varietà dell'analisi.
Grazie a questo lavoro, dopo aver studiato questi algoritmi da un punto di vista teorico, si valuta la loro efficienza empirica tramite misure ripetute dei tempi di esecuzione su input casuali e, quindi, si confermano o smentiscono le previsioni teoriche legate alla loro complessità. Sono state effettuate analisi dettagliate al variare della lunghezza n dell'array e dell'intervallo dei valori m, utilizzando per ciascun parametro dei campioni in successione geometrica.

2 Algoritmi di Ordinamento

2.1 *Quick Sort*

Quick Sort è un algoritmo che si basa sul concetto del *divide et impera*. L'idea di base è quella di scegliere, all'interno di un array di n elementi, un *pivot* e di spostare gli elementi più piccoli prima e quelli più grandi dopo di esso. La scelta del pivot può essere effettuata in diverse posizioni dell'array: infatti, può essere il primo elemento, l'ultimo, quello centrale oppure un elemento casuale. La scelta del pivot è fondamentale poiché incide sull'efficienza di questo algoritmo. Applicando quindi ricorsivamente lo stesso algoritmo alle due partizioni dell'array risultanti, si ottiene un array ordinato.
L'algoritmo `Partition` qui implementato non segue né la versione di Hoare, creatore di *Quick Sort* (1960), né quella di Lomuto, ma come pivot sceglie l'elemento centrale.
La complessità di questo algoritmo nel caso peggiore è di $\Theta(n^2)$, ma è raro che si verifichi; nel caso medio *Quick Sort* ha complessità $\Theta(n \log n)$. Guardando i suoi costi ci si potrebbe chiedere il perché del nome se, apparentemente, sembra essere costoso tanto quanto, ad esempio, *Merge Sort*. La ragione della sua efficienza risiede nelle costanti moltiplicative: infatti, quelle di *Quick Sort* sono decisamente inferiori rispetto a quelle degli altri algoritmi, rendendolo mediamente più veloce.

2.2 *Quick Sort 3-way*

Quick Sort 3-way estende la versione classica andando a introdurre un terzo partizionamento in modo da poter distinguere gli elementi minori, uguali e maggiori del pivot. L'algoritmo, infatti, dopo aver selezionato il pivot più adatto, scansiona l'array al fine di classificare gli elementi nel modo indicato in precedenza. Questa procedura si fermerà nel momento in cui il puntatore che scandisce l'array avrà superato il puntatore del confine inferiore della parte contenente gli elementi maggiori del pivot. Si avrà quindi l'array diviso nelle tre parti. *Quick Sort 3-way* applicherà poi lo stesso procedimento esclusivamente alla parte contenente gli elementi maggiori e a quella dei minori del pivot. Per quanto riguarda gli elementi uguali al pivot, non è necessario effettuare ulteriori passaggi, poiché risultano già ordinati per definizione.
Questa versione è particolarmente vantaggiosa a livello computazionale, in modo particolare quando si è in presenza di un array contenente numerosi duplicati: l'algoritmo riduce il numero di chiamate ricorsive e dei confronti, in quanto evita di riordinare gli elementi uguali al pivot.

2.3 *Counting Sort*

Counting Sort è un algoritmo che si differenzia dagli algoritmi precedentemente analizzati in quanto non si basa su operazioni di scambio e confronti tra gli elementi. Per essere efficiente, e quindi avere un costo computazionale di $\Theta(n)$, deve soddisfare un'ipotesi aggiuntiva sull'input che gli altri algoritmi di ordinamento non hanno. Nella versione classica dell'algoritmo l'ipotesi è la seguente: gli elementi dell'array devono essere interi, positivi e appartenenti ad un determinato intervallo $[0, k]$. Nonostante le specifiche del progetto non lo richiedessero esplicitamente, è stata implementata una versione estesa di *Counting Sort* in grado di supportare anche valori interi negativi. Questa scelta progettuale è stata effettuata al fine di garantire una maggiore robustezza e flessibilità dell'algoritmo implementato, permettendo così di superare con successo tutti i casi di test previsti dall'ambiente di valutazione.
L'algoritmo fa uso di due strutture ausiliarie: un array B di lunghezza n, che sarà il vettore di output nel quale verranno copiati gli elementi ordinati (inizialmente verrà popolato progressivamente durante l'esecuzione dell'algoritmo); e un array C che fungerà da accumulatore, la cui dimensione dipenderà dall'ampiezza del range dei valori. In questo vettore, in posizione $C[h]$ si troverà il numero che descrive la quantità di elementi minori o uguali a h. Terminata l'operazione di accumulo, si procede scandendo il vettore A da destra verso sinistra per copiare in B gli elementi di A nella giusta posizione senza perdere dati importanti.

2.4 *Merge Sort*

L'idea di base di *Merge Sort* è quella di dividere l'array in due porzioni di ugual misura, oppure il più possibile di dimensioni simili, utilizzando il floor della lunghezza dell'array diviso per due. Così facendo si avranno due sottoproblemi che verranno ordinati ricorsivamente. Una volta ottenute le soluzioni dei due sottoproblemi, l'algoritmo richiama la procedura `Merge` che si occupa di unire le due porzioni e fonderle in un unico array ordinato. Il `Merge` procede utilizzando due puntatori, ciascuno posizionato all'inizio dei due array ordinati, al fine di confrontare, ad ogni iterazione, gli elementi puntati e copiare l'elemento minore nell'array ausiliario. Questa procedura funziona correttamente solo se viene soddisfatta la sua precondizione: le due porzioni date come input devono essere ordinate. Inoltre, rende *Merge Sort* un algoritmo non *in-place* in quanto utilizza un array ausiliario per memorizzare i valori ordinati.
La complessità totale è di $\Theta(n \log n)$, mentre quella di `Merge` è di $\Theta(n)$.

3 Algoritmi di Misurazione

3.1 Resolution

Resolution è una funzione fondamentale che permette di determinare empiricamente la risoluzione effettiva del timer ad alta precisione del sistema (clock monotono) fornito dal sistema operativo. Questa risoluzione rappresenta il più piccolo intervallo di tempo che il timer è in grado di rilevare in maniera affidabile, e varia sensibilmente in base alle caratteristiche dell'hardware del computer e del sistema operativo utilizzati per le misurazioni.
La funzione procede registrando il tempo iniziale andando a richiamare il clock di sistema. Successivamente avvia un ciclo continuo che monitora il tempo attuale, facendo ripetute chiamate al timer, finché questo non differisce dal valore iniziale che era stato registrato. Quando viene rilevata la differenza, il valore viene registrato come tempo finale. Infine, restituisce la differenza tra il tempo finale e quello iniziale, che corrisponde alla risoluzione effettiva del clock.
È importante avere una conoscenza precisa di questa risoluzione perché garantisce di avere delle misure accurate e affidabili nelle fasi successive. Questo perché permette di determinare l'intervallo minimo di tempo da utilizzare nella misurazione degli algoritmi di ordinamento assicurando la precisione voluta, in questo caso $\leq 0{,}001$.

3.2 Measure

Measure è stato progettato con lo scopo di misurare il tempo medio di esecuzione di un algoritmo in maniera statisticamente affidabile. È stato pensato per avere il minimo impatto dovuto dalle fluttuazioni casuali (derivanti da fattori esterni come carichi del sistema operativo, caching o operazioni secondarie) per garantire una misura coerente e riproducibile.
L'algoritmo procede andando a leggere i parametri da testare: n (la dimensione dell'input), m (il valore massimo per la generazione casuale dei dati) e infine T_{min} (il tempo minimo di misura calcolato dalla procedura `Resolution`). Successivamente salva il valore del clock per registrare il tempo iniziale. In seguito, entra in un ciclo in cui ad ogni iterazione viene: generato un array casuale di n elementi compresi nell'intervallo $[1, m]$, viene eseguito l'algoritmo di ordinamento sull'array appena generato, viene incrementato il contatore che tiene traccia del numero totale di iterazioni effettuate e si verifica il tempo trascorso. Il ciclo si interrompe appena il tempo totale accumulato non raggiunge o supera il tempo minimo prefissato e, a questo punto, registra il tempo finale. L'algoritmo restituisce il tempo medio di esecuzione, calcolato come rapporto tra il tempo accumulato e il numero totale di iterazioni effettuate.
È importante l'utilizzo di questa strategia di misurazione per due motivi principali: l'accuratezza della misura e l'affidabilità statistica. Si ottiene l'accuratezza nelle misurazioni eseguendo più iterazioni fino al raggiungimento di un intervallo temporale predefinito che permette di ridurre significativamente l'effetto delle fluttuazioni casuali, in modo da rendere la stima del tempo medio più stabile e affidabile rispetto a misurazioni singole. L'affidabilità statistica, invece, si ottiene prendendo numerosi campioni di input diversi (infatti si generano array casuali ad ogni iterazione) e, di conseguenza, si ottiene una misura media che rappresenta fedelmente il comportamento generale dell'algoritmo.

4 Rappresentazione Grafica dei Risultati

Partiamo con l'analisi dei due grafici in funzione di n:

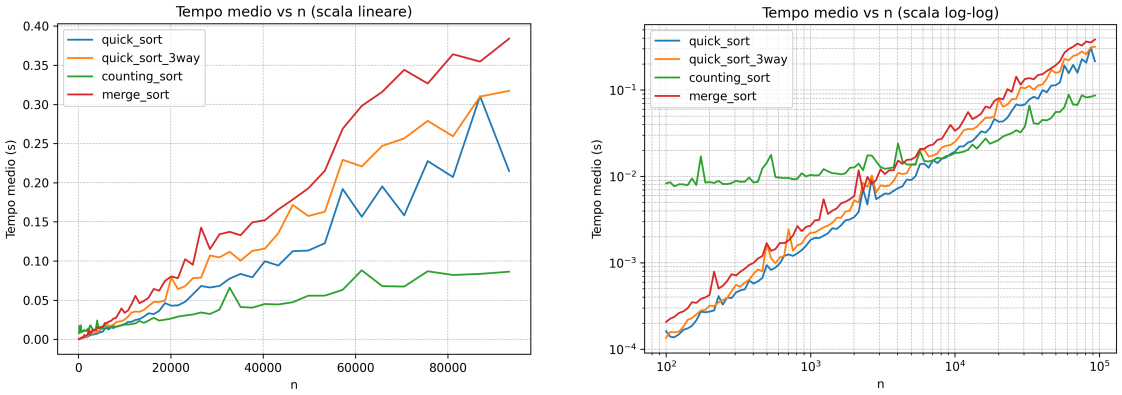


Figura 1: Grafici dei tempi di esecuzione in funzione di n (scala lineare a sinistra, scala log–log a destra).

I grafici ottenuti fissando il valore di $m = 100\,000$ e variando la lunghezza dell'array n confermano in modo molto accurato le previsioni teoriche circa le complessità degli algoritmi analizzati.

- *Counting Sort*, avendo una complessità teorica di $O(n + m)$, risulta inizialmente il più veloce con valori piccoli e medi di n, grazie alla dominanza del termine lineare rispetto al costo logaritmico più elevato degli altri algoritmi. Tuttavia, si può vedere che, al crescere di n, mantiene un andamento riconducibile a quello lineare, ma il costo assoluto peggiora e si avvicina gradualmente a quello degli algoritmi basati su scambi e confronti. In ogni caso rimane comunque il più efficiente.

- *Quick Sort* classico e *Quick Sort 3-way* mostrano entrambi un chiaro andamento di tipo $O(n \log n)$, coerente con la loro analisi teorica. Dai grafici emerge che *Quick Sort* classico risulta leggermente più performante rispetto a *Quick Sort 3-way*, il quale introduce un piccolo overhead dovuto alla gestione aggiuntiva dei puntatori necessari per partizionare in tre zone distinte. Ricordiamo che *Quick Sort 3-way* è progettato specificatamente per trattare in modo ottimale input con molti duplicati, quindi il leggero svantaggio riscontrato in questo caso era prevedibile.

- *Merge Sort* si colloca sempre sopra le curve delle due versioni di *Quick Sort*, mostrando tempi mediamente più elevati. Questa differenza di performance è attribuibile alla necessità di allocare memoria aggiuntiva e ai costi derivanti dalla copia dei sotto-array in ogni passaggio della procedura di *Merge*. Tuttavia, l'andamento generale del grafico conferma la sua complessità.

L'analisi in scala logaritmica (log–log) rende ancora più evidente quanto appena osservato: *Counting Sort* presenta una pendenza prossima a 0, caratteristica della complessità lineare, mentre i tre algoritmi basati su confronti e scambi presentano degli andamenti paralleli con pendenze tendenti a 1, confermando in modo chiaro la loro complessità teorica $O(n \log n)$.

Passiamo ora all'analisi in funzione di m:

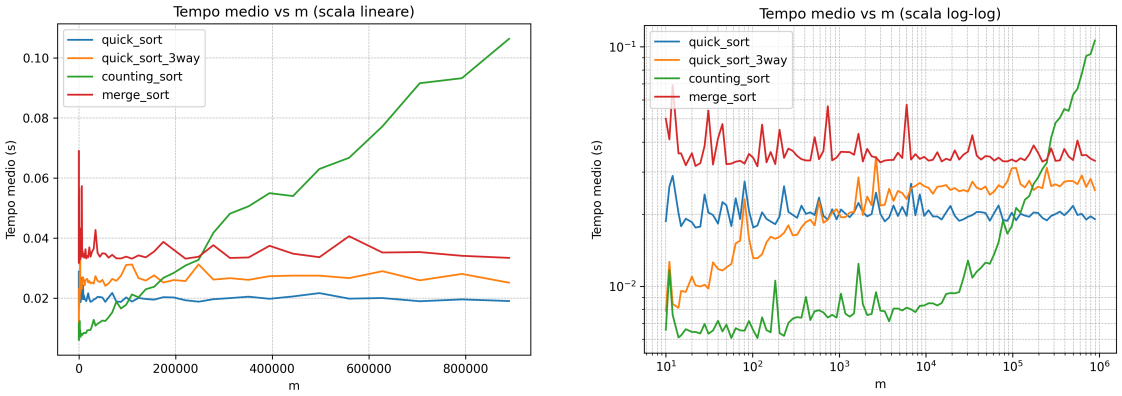


Figura 2: Grafici dei tempi di esecuzione in funzione di m (scala lineare a sinistra, scala log–log a destra).

- Dai risultati ottenuti emerge chiaramente come *Counting Sort* sia l'unico algoritmo fortemente sensibile alle variazioni di m, che lo portano a un sensibile peggioramento delle prestazioni. Nei grafici, *Counting Sort* mostra una crescita del tempo lineare con l'aumento di m, confermando la sua complessità teorica $O(n + m)$. In particolare, nel grafico in scala lineare si osserva una crescita quasi perfettamente lineare, partendo da tempi molto bassi con valori piccoli di m (pochi millisecondi) e arrivando a tempi nettamente più elevati (oltre 0,1 secondi) quando m raggiunge valori prossimi al milione.

- Per quanto riguarda *Quick Sort 3-way*, possiamo notare nel grafico con scala logaritmica come i tempi siano significativamente più bassi quando i valori di m sono in un range limitato e, di conseguenza, l'input contiene molti duplicati. Questo era stato evidenziato in precedenza nel paragrafo sui grafici legati alle variazioni di n.

- *Quick Sort* classico, *Quick Sort 3-way* e *Merge Sort* presentano tempi praticamente costanti al variare del range degli interi, con oscillazioni dovute probabilmente a fattori esterni non controllabili direttamente avvenuti durante la misurazione (si possono notare soprattutto nel grafico a scala lineare). Questa osservazione è coerente con le aspettative teoriche, poiché questi algoritmi basano il loro comportamento unicamente sulla lunghezza dell'array e non sul valore massimo contenuto nell'array stesso.

5 Conclusioni

Grazie a questo progetto si è presentata l'occasione per verificare empiricamente la validità delle aspettative poste dallo studio teorico riguardo alla complessità temporale degli algoritmi di ordinamento implementati: *Quick Sort*, *Quick Sort 3-way*, *Counting Sort* e *Merge Sort*. Le analisi effettuate in maniera sperimentale, e svolte con rigore scientifico, hanno permesso di confrontare efficacemente la teoria con la pratica evidenziando la coerenza delle complessità teoriche con i risultati empirici.
I risultati ottenuti confermano chiaramente i modelli teorici. Infatti, gli algoritmi *Quick Sort* e *Merge Sort* hanno mostrato un andamento molto vicino a quello previsto dalla complessità teorica di $O(n \log n)$. *Quick Sort 3-way*, variante specificatamente progettata per gestire array con un numero elevato di elementi duplicati, ha confermato la sua efficienza soprattutto nel caso di valori limitati del range. *Counting Sort* ha dimostrato la propria efficacia quando il range dei dati è fissato, mentre c'è un significativo aumento del tempo di esecuzione al crescere del parametro m.
L'approfondimento dettagliato dei risultati ha permesso, inoltre, di identificare situazioni applicative specifiche per ciascun algoritmo, offrendo importanti linee guida per la selezione ottimale delle tecniche algoritmiche in scenari concreti:

- *Quick Sort* si è dimostrato un algoritmo versatile ed efficace per dataset generici senza particolari disturbi nei dati, rappresentando quindi una scelta valida per la maggior parte dei contesti generali.
- *Quick Sort 3-way* ha evidenziato chiaramente il proprio vantaggio computazionale nelle situazioni in cui ci sono frequenti duplicati, consentendo una riduzione significativa del numero delle chiamate ricorsive.
- *Counting Sort* è risultato nettamente superiore in situazioni con range fissato (e con m relativamente basso), confermandosi una scelta ideale per problemi in cui i dati sono interi limitati entro intervalli specifici.
- *Merge Sort* pur evidenziando, a livello teorico, una complessità simile a *Quick Sort*, ha mostrato di essere utile in contesti in cui la stabilità dell'ordinamento è un requisito necessario.

L'adozione di strumenti automatici per la raccolta, analisi e rappresentazione dei dati si è dimostrata una scelta idonea. Questa metodologia ha semplificato il processo di valutazione e ha assicurato un elevato grado di precisione e chiarezza nei risultati ottenuti. La generazione dei grafici ha permesso una visualizzazione immediata e intuitiva delle tendenze degli algoritmi, facilitando l'interpretazione dei risultati sperimentali.

In conclusione, confermando le aspettative teoriche sulle complessità algoritmiche, questo progetto si può definire ben riuscito; in aggiunta sottolinea quanto sia importante la scelta degli algoritmi in base alle caratteristiche del problema e dei dati. Il lavoro svolto offre una metodologia robusta e replicabile, rappresentando un punto di partenza per analisi algoritmiche future.